Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

Materia: Implementación de robótica inteligente

Clave: TE3002B.501

Grupo: 501

Profesor:
Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios
Alfredo García Suárez
Cesar Torres Huitzil

Alumnos
Omar Perez Dominguez | A01738306
Ezequiel Luna Trejo | A01738153
Antonio Méndez Rodríguez | A01738269

12 de Abril De 2026

Resumen

En el siguiente reporte documenta el desarrollo y ejecución del Challenge de la semana 2, centrado en el diseño de un controlador de lazo abierto robusto y auto-ajustable para el robot Puzzlebot. Para poder realizar este reto tuvimos que hacer la identificación de la zona muerta de los motores, hasta la implementación de un nodo generador de trayectorias multiobjetivo.

El proceso consistió en programar una Máquina de Estados Finitos (FSM) para gestionar transiciones precisas entre desplazamientos lineales y giros , permitiendo al robot completar diferentes trayectorias. Un elemento notable fue la integración de un algoritmo de validación de factibilidad, donde el sistema estima automáticamente velocidades y tiempos requeridos, informando al usuario si los objetivos son alcanzables según la dinámica real del robot.

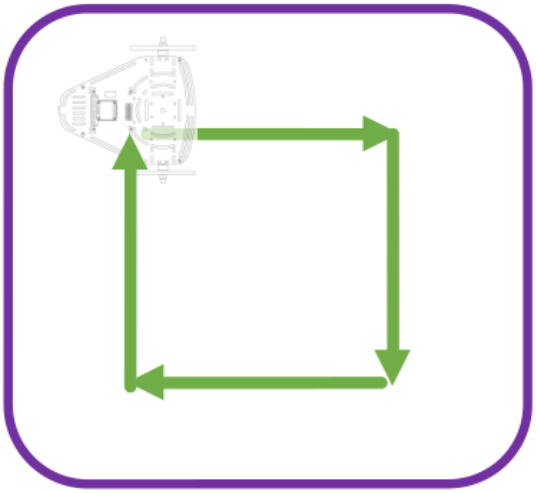

Imagen 1.Ejemplo de una trayectoria del robot

Objetivos

Diseñar e implementar un sistema de navegación de lazo abierto en ROS 2 para el robot Puzzlebot que integre la ejecución de trayectorias geométricas fijas y la navegación hacia coordenadas dinámicas, asegurando la robustez operativa y la validación para lograr esas coordenadas dinámicas.

Objetivos específicos

- Desarrollar un nodo de control para ejecutar una trayectoria cuadrada de 2 metros por lado realizada por el robot.

- Implementar una lógica de control "auto-ajustable" que permita al sistema estimar velocidades y tiempos de ejecución a partir de las coordenadas de navegación ingresadas por el usuario y el tiempo.
- Programar un generador de trayectorias que procese una lista de puntos objetivos y determine si cada destino es alcanzable considerando las capacidades dinámicas reales del sistema.
- Definir un mensaje personalizado para el tópico /pose que extienda la funcionalidad de los mensajes estándar para incluir información crítica sobre tiempos de llegada y velocidades de referencia.

Introducción

El desarrollo de la robótica móvil autónoma requiere la integración de modelos matemáticos precisos y protocolos de comunicación estandarizados que permitan traducir objetivos espaciales en movimientos físicos. En el contexto de este reto, se transita del control manual hacia la ejecución de trayectorias predefinidas mediante el uso del framework ROS 2 y el robot diferencial Puzzlebot. Para fundamentar esta investigación, es necesario comprender los siguientes conceptos:

Mensajes twist y pose en ROS 2:

- Mensajes twist (geometry_msgs/Twist): Son los comandos de control que dictan la velocidad instantánea del vehículo. Estos mensajes se componen de dos vectores principales: la velocidad lineal (en metros por segundo) y la velocidad angular (en radianes por segundo). La Hackerboard recibe estos datos y los procesa para determinar el voltaje que debe aplicarse a cada rueda .
- Mensajes pose (geometry_msgs/Pose): Definen la ubicación espacial del robot en un plano cartesiano. Para los fines de este reto, se diseñó un mensaje personalizado que no solo transporta la posición (x, y) y la orientación, sino que también integra parámetros de tiempo y velocidad solicitados por el usuario, permitiendo que el controlador tenga una visión completa del objetivo antes de iniciar el movimiento

Lazo abierto de control

Un sistema de lazo abierto es aquel en el que la acción de control se calcula basándose exclusivamente en un modelo matemático y en el tiempo, sin utilizar retroalimentación de sensores externos para corregir errores en tiempo real. En el caso del Puzzlebot, esto implica que el robot ejecuta una secuencia de estados basada en cálculos previos. Para que este control sea robusto frente a perturbaciones y no linealidades, el controlador debe ser capaz de auto-ajustarse según las preferencias de velocidad o tiempo ingresadas por el usuario.

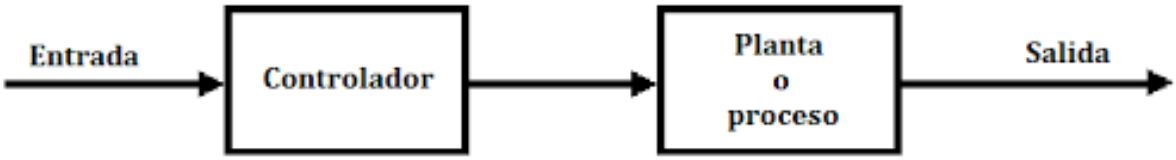

Imagen 2. Diagrama de un sistema de control a lazo abierto

Caracterización de motores: zona muerta y saturación

Para que un controlador de lazo abierto sea funcional, es imperativo identificar la región lineal de operación de los motores, para esto necesitamos:

- Zona muerta (dead zone): Es el rango de velocidad mínimos en los cuales el motor no logra vencer la fricción estática interna y el robot permanece inmóvil. Identificar este valor permite que el código aplique siempre una potencia suficiente para iniciar el movimiento.
- Saturación: Es el punto en el que, aunque se aumente el comando de velocidad en el software, el motor ha alcanzado su límite físico de revoluciones por minuto debido al voltaje máximo de la batería. Conocer este límite evita que el controlador intente ejecutar trayectorias físicamente imposibles.

Máquinas de estados finitos (FSM) en robótica

Una máquina de estados finitos es un modelo computacional que permite estructurar el comportamiento del robot de manera lógica y modular . En lugar de tener un código lineal difícil de depurar, el comportamiento se divide en estados discretos (como "Girar", "Avanzar" o "Detenerse") . El robot transita entre estos estados basándose en condiciones específicas, como el cumplimiento de un temporizador o el alcance de un objetivo. Esta estructura asegura que el código sea escalable y fácil de mantener .

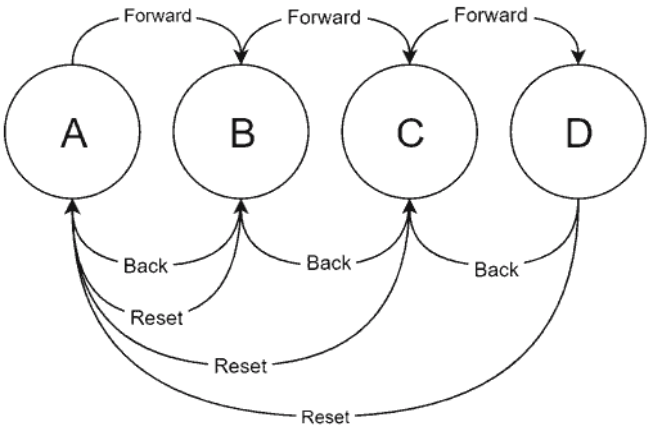

Imagen 3. Diagrama de una máquina de estados

Cálculo de navegación: distancia y ángulo

Para que el robot navegue hacia un objetivo de manera autónoma, el controlador debe realizar cálculos cinemáticos básicos:

- Cálculo de distancia: Se emplea la fórmula de la distancia euclidiana entre la posición inicial y el punto destino (x, y), lo que determina el tiempo total que el robot debe avanzar a una velocidad lineal constante.
- Cálculo de ángulo: Se utiliza la función atan2(y, x) para determinar la orientación necesaria. El robot debe primero girar sobre su propio eje hasta alinearse con este ángulo antes de iniciar el desplazamiento lineal.

Solución del problema

En esta sección se detalla la metodología técnica empleada para cumplir con los objetivos del reto, integrando la caracterización física del hardware con la arquitectura de software desarrollada en ROS 2 Humble.

Metodología de calibración y caracterización

Para lograr que el robot recorra distancias y realice giros de forma autónoma y precisa, se adoptó un enfoque empírico, pasando de un modelo puramente teórico a uno basado en la experimentación directa en el laboratorio. El proceso se centró en determinar la mecánica real del Puzzlebot a través de una secuencia de pruebas y ajustes.

Validación dinámica del movimiento

Para garantizar que el robot pueda cumplir con las trayectorias solicitadas por el usuario, se implementó un mecanismo de validación dinámica basado en la caracterización experimental de los motores. En un esquema de control en lazo abierto, es fundamental asegurar que los tiempos y velocidades requeridos sean físicamente alcanzables, ya que no existe retroalimentación que permita corregir errores durante la ejecución.

Como parte de la caracterización del sistema, se determinaron los límites operativos de los motores:

- Zona muerta (dead zone): Se identificó que el robot comienza a responder a partir de aproximadamente 0.0004654 m/s en velocidad lineal y 0.01673 rad/s en velocidad angular. Por debajo de estos valores, el robot no presenta movimiento debido a efectos como la fricción estática y limitaciones propias de los actuadores.
- Saturación: Se determinó que las velocidades máximas dentro de una región de comportamiento aproximadamente lineal son 0.3145 m/s para la velocidad lineal y 3.99 rad/s para la velocidad angular. Más allá de estos valores, el sistema entra en una zona de saturación donde los incrementos en la señal de entrada no se traducen proporcionalmente en movimiento.

Con base en estos resultados, se estableció un criterio de validación donde se calcula el tiempo mínimo necesario para completar cada segmento de la trayectoria, considerando tanto la rotación como el desplazamiento lineal. Este cálculo utiliza como referencia las velocidades máximas reales del sistema (saturación), asegurando que el robot opere dentro de su región funcional.

De esta manera, si el usuario solicita un tiempo menor al físicamente posible, el sistema rechaza la instrucción, evitando comportamientos no realizables o trayectorias incompletas. Asimismo, el diseño del controlador considera que las velocidades efectivas deben mantenerse por encima de la zona muerta para garantizar el movimiento.

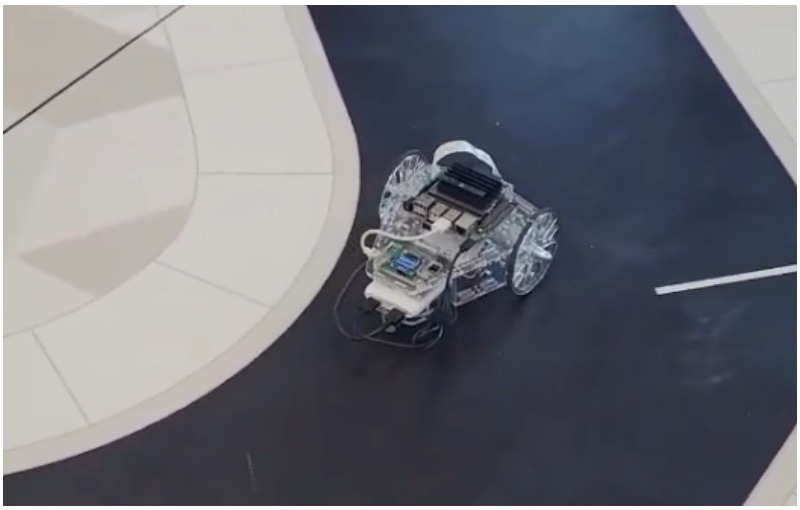

Imagen 4. Puzzlebot realizando pruebas

Estrategia de control en lazo abierto basada en segmentación

El control del robot se diseñó bajo un enfoque de lazo abierto, en el cual no se utiliza retroalimentación sensorial para corregir la trayectoria durante el movimiento. En lugar de implementar el modelo cinemático completo mediante integración continua, se optó por una simplificación basada en la segmentación del movimiento.

Cada trayectoria se divide en dos etapas principales:

- Rotación: el robot ajusta su orientación hasta alinearse con el objetivo.
- Traslación: el robot avanza en línea recta hasta alcanzar la posición deseada.

Esta estrategia se fundamenta en relaciones físicas básicas, donde el desplazamiento lineal depende de la velocidad y el tiempo, y el cambio de orientación depende de la velocidad angular y el tiempo. Esta simplificación permite implementar un control eficiente y computacionalmente sencillo, adecuado para sistemas sin retroalimentación.

Calibración experimental del sistema

Durante las pruebas iniciales, se observó que el comportamiento real del robot difería significativamente del esperado teóricamente. En particular, al ejecutar una trayectoria de referencia (avance de 2 metros y giro de 180°), el robot tendía a:

- Recorrer distancias mayores a las esperadas
- Realizar giros incompletos

Estas discrepancias se deben a factores físicos no modelados, como el deslizamiento de las ruedas, la fricción y la dinámica propia de los motores. Para corregir estos errores, se realizó una serie de experimentos controlados, variando las velocidades lineales y angulares, y midiendo los resultados reales obtenidos. Los datos recolectados fueron los siguientes:

Velocidad angular constante, linear variable		
Distancia recorrida	Velocidad lineal	Velocidad angular
2.11m	0.2	0.5
2.18m	0.15	0.5
2.17m	0.1	0.5

Velocidad lineal constante, angular variable		
Giro en grados	Velocidad angular	Velocidad lineal
152º	0.5	0.1
160º	0.7	0.1
140º	1.4	0.1
155º	0.8	0.1

	Distancia recorrida	Giro en grados	Velocidad lineal	Velocidad angular
Promedios	2.155	151	0.16	0.85
Esperado	2	180	0.1488	1.0087

Imagen 5. Registro de los datos obtenidos en las pruebas

A partir de estos datos, se calcularon valores promedio que permitieron caracterizar el comportamiento real del sistema:

- Distancia promedio: 2.155 m
- Giro promedio: 151º

En comparación con los valores esperados:

- Distancia teórica: 2 m
- Giro teórico: 180º

Ajuste del control mediante calibración

Con base en los resultados experimentales obtenidos, se realizó un proceso de calibración con el objetivo de corregir las diferencias entre el comportamiento teórico del robot y su

desempeño real. Durante las pruebas iniciales, se observó que, aun cuando se calculaban correctamente las trayectorias usando relaciones ideales de movimiento, el robot no cumplía con los resultados esperados: tendía a recorrer más distancia de la indicada y a girar menos del ángulo deseado.

Para entender y corregir este comportamiento, se realizaron múltiples experimentos en los que se fijaban ciertas velocidades de entrada y se medía el resultado real del robot, tanto en desplazamiento como en rotación. A partir de estos datos, se identificó que existe una discrepancia sistemática entre las velocidades que se le "piden" al robot y las que realmente ejecuta.

En lugar de modificar directamente el modelo de movimiento, se optó por una estrategia de compensación. Esta consiste en ajustar las velocidades calculadas teóricamente antes de enviarlas al robot, utilizando como referencia la relación entre:

- Los valores de velocidad que se envían como comando
- Las velocidades máximas reales que el robot es capaz de alcanzar

A partir de esta relación, se construyó un factor de ajuste que corrige proporcionalmente las velocidades requeridas para cada movimiento. Es decir, primero se calcula qué velocidad sería necesaria en un sistema ideal para cumplir con la trayectoria en el tiempo deseado, y posteriormente esa velocidad se reduce o ajusta considerando el comportamiento real observado en el robot.

Este procedimiento permite compensar errores como el exceso de desplazamiento o los giros incompletos, sin necesidad de modificar la estructura del controlador. Además, al tratarse de un ajuste proporcional, este se adapta automáticamente a distintas trayectorias, en lugar de depender de valores fijos.

Estructura del mensaje personalizado TargetPose

Para dotar al sistema de mayor flexibilidad, se implementó una interfaz propia mediante el mensaje TargetPose.msg. Esta estructura permite que el nodo reciba y publique información relevante sobre cada punto objetivo de la trayectoria.

El mensaje se compone de los siguientes parámetros:

- x / y: Coordenadas cartesianas del destino
- req_time: Tiempo requerido para completar el movimiento
- req_v: Velocidad lineal aplicada
- req_w: Velocidad angular aplicada

Esta personalización permite monitorear y validar que las condiciones de movimiento sean consistentes con las restricciones dinámicas del sistema solicitado es factible dentro de los límites de saturación previamente identificados.

Arquitectura del nodo y lógica de programación

La implementación se realizó mediante el nodo trajectory_controller, cuya función principal es transformar las metas espaciales ingresadas por el usuario en comandos de velocidad del tipo Twist, los cuales se publican en el tópico /cmd_vel para controlar el movimiento del robot.

El nodo sigue una arquitectura basada en ROS 2, en la que se emplean publishers para enviar información tanto de control como de monitoreo. En particular, se utilizan dos tópicos principales:

- /cmd_vel: para enviar las velocidades lineales y angulares del robot.
- /pose: para publicar información del objetivo actual mediante el mensaje personalizado TargetPose.

El funcionamiento del nodo inicia solicitando al usuario una serie de puntos objetivo, definidos por coordenadas en el plano y un tiempo deseado para alcanzar cada uno. A partir de estos datos, el sistema calcula la distancia y el ángulo necesarios para llegar al objetivo, y posteriormente realiza una validación dinámica para verificar que el movimiento sea físicamente posible.

Una vez validado el punto, el nodo calcula las velocidades requeridas para cumplir con el tiempo especificado y aplica un factor de calibración para ajustar dichas velocidades al comportamiento real del robot. Estos valores son almacenados en una estructura interna que permite gestionar múltiples puntos de trayectoria de forma secuencial.

La ejecución del movimiento se controla mediante un temporizador que actualiza periódicamente el estado del sistema. En cada iteración, el nodo determina la acción a realizar (giro o avance) y publica los comandos de velocidad correspondientes.

Adicionalmente, el nodo publica información del objetivo actual, incluyendo posición, tiempo requerido y velocidades aplicadas, lo que permite monitorear el desempeño del sistema durante la ejecución.

Seguimiento del código y máquina de estados (FSM)

El flujo del programa está gestionado por una máquina de estados finitos (FSM) que asegura una ejecución secuencial y controlada. La lógica se divide en los siguientes estados operativos:

- Estado 0 (Inicialización): Se selecciona el siguiente punto objetivo y se preparan los parámetros del movimiento.
- Estado 1 (Giro): El robot rota hasta alinearse con la dirección del objetivo.
- Estado 2 (Avance): El robot se desplaza en línea recta hacia la posición objetivo.
- Estado 3 (Finalización): Se detiene el robot al completar todos los puntos de la trayectoria.

Resultados

Como prueba principal, se implementó una trayectoria en forma de cuadrado de dimensiones 2*2 m. El robot fue capaz de recorrer correctamente los cuatro segmentos de la trayectoria, alcanzando cada uno de los puntos objetivo de manera consistente. Asimismo, al finalizar el recorrido, el robot regresó al punto de inicio con una orientación cercana a la original.

Se observó un error de posicionamiento aproximado de 1 cm, lo cual representa una desviación mínima considerando que el sistema opera en lazo abierto. Este error puede atribuirse a factores como el deslizamiento de las ruedas, pequeñas variaciones en la superficie y la acumulación de errores a lo largo de la trayectoria.

Adicionalmente, se realizaron pruebas con trayectorias más complejas, incluyendo puntos ubicados en el eje negativo del plano cartesiano. En estos casos, el robot también fue capaz

de calcular correctamente la orientación requerida y ejecutar los movimientos necesarios para alcanzar los objetivos, demostrando la capacidad del sistema para manejar cambios de dirección y diferentes configuraciones espaciales.

Los resultados obtenidos evidencian que la metodología de calibración y el ajuste de velocidades implementados permiten mejorar significativamente la precisión del sistema, logrando un comportamiento consistente y repetible en distintas condiciones de prueba.

Ejecucion_de_trayectoria.mp4

Conclusiones

Tras la culminación del Mini Reto 1, se presenta el análisis de los resultados y el nivel de cumplimiento de los objetivos establecidos para esta etapa de investigación y desarrollo.

Análisis del cumplimiento de los objetivos

Se considera que los objetivos del reto se lograron satisfactoriamente por las siguientes razones:

- Ejecución de trayectorias: El nodo desarrollado fue capaz de conducir al Puzzlebot en una trayectoria cuadrada de 2 metros por lado de manera repetible.
- Implementación auto-ajustable (Auto-tuned): Se logró que el controlador estimará de forma automática los tiempos y velocidades necesarias basándose en los parámetros ingresados por el usuario, cumpliendo con los puntos dinámicos y no fijos.
- Validación de factibilidad: El sistema integra una lógica de seguridad que informa al usuario si un punto es inalcanzable, para poder lograr esto tuvimos que estudiar el comportamiento dinámico del robot y sus límites físicos .

Logros y robustez del sistema

El éxito en el cumplimiento de los objetivos se debe primordialmente a dos factores: la caracterización previa de los motores (identificando la zona muerta y la saturación) y el uso de una Máquina de Estados Finitos (FSM) . Esta estructura permitió que el robot transitara de forma suave entre estados de giro y avance, minimizando el impacto de las no linealidades de los actuadores.

Sin embargo, es importante señalar que los objetivos no se cumplieron con una precisión absoluta en el cierre de la trayectoria en el robot real. Esto se debe a la naturaleza del control

en lazo abierto, el cual es imposible evitar el ruido y las perturbaciones externas como el deslizamiento de las llantas en el piso del laboratorio o pequeñas variaciones en el voltaje de la batería. Al no contar con retroalimentación sensorial (encoders), el error de posición se acumuló a lo largo de los cuatro lados del cuadrado, también esto lo notamos cuando el robot realizaba una ruta diferente al cuadrado.

Como solución y mejora se propone lo siguiente:

- Transición a lazo cerrado: Implementar un control basado en odometría que utilice los sensores del robot como los encoders para corregir la trayectoria en tiempo real, compensando la fricción y asegurando que el robot llegue exactamente a la posición deseada.
- Compensación dinámica por nivel de batería: Dado que el control de lazo abierto depende de la potencia entregada a los motores, se sugiere añadir un factor de corrección basado en el voltaje de la batería. Esto permitiría ajustar los tiempos de ejecución de forma dinámica para mantener la velocidad real constante a pesar de la descarga natural de la fuente de energía.

Referencia:

Manchester Robotics Ltd. (2026). TE3002B: Intelligent Robotics Implementation [Repositorio de GitHub]. Recuperado de https://github.com/ManchesterRoboticsLtd/TE3002B_Intelligent_Robotics_Implementation_2026

micro-ROS. (2026). micro-ROS: ROS 2 on microcontrollers. Recuperado de https://micro.ros.org/

Open Robotics. (2026). ROS 2 Documentation: Humble Hawksbill. Recuperado de https://docs.ros.org/en/humble/